Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа №7 по дисциплине «Экономика программной инженерии»

Тема Оценка параметров программного проекта с использованием модели COCOMO II

Студент Шахнович Дмитрий Сергеевич

Группа ИУ7-82Б

Преподаватель Барышникова М.Ю.

Москва, 2026

Введение

Целью лабораторной работы №7 является продолжение знакомства с существующими методиками предварительной оценки параметров программного проекта и практическая оценка затрат по модели COCOMO II.

Ход работы

Описание варианта задания

Компания получила заказ на разработку клиентского мобильного приложения брокерской системы. Программа позволяет просматривать актуальную биржевую информацию, производить сделки и отслеживать их выполнение. Приложение имеет 4 страницы: авторизация, биржевые сводки, заявки, новая заявка.

Авторизация

На данной странице осуществляется ввод логина и пароля пользователя для входа в систему. Страница содержит два поля ввода и одну командную кнопку, а также флажок-переключатель, который активируется при необходимости запоминания параметров авторизации.

Биржевые сводки

Биржевые сводки отражают текущую ситуацию на бирже. Страница содержит таблицу, кнопку «Добавить» и диалоговое окно с одним полем для ввода и двумя командными кнопками.

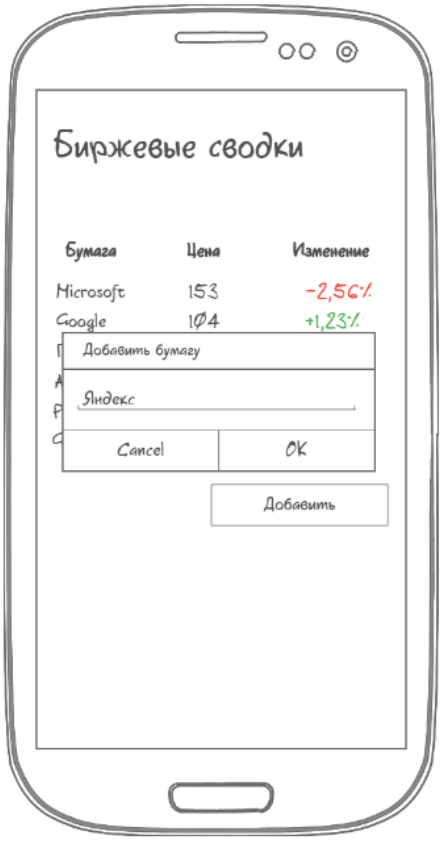


Таблица содержит три колонки: Ценная бумага (имя бумаги), Цена (цена за одну ценную бумагу), Изменение (изменение цены бумаги со времени последнего закрытия биржи). Кнопка «Добавить» вызывает диалоговое окно для добавления новой бумаги (окно состоит из поля ввода и кнопок OK, Cancel).

Заявки

Заявки содержат таблицу, отображающую текущие (еще не выполненные) заявки на покупку или продажу ценных бумаг. Таблица содержит четыре поля: Тип (покупка/продажа), Имя бумаги, Цена по которой готовы покупаться/продаваться бумаги, Количество бумаг для покупки/продажи. При нажатии на любую строку таблицы появляется контекстное меню с возможностью удалить или изменить заявку.

Новая заявка

Страница позволяет оформить заявку на покупку или продажу ценной бумаги. Страница состоит из 4 полей: Бумага (имя бумаги), Цена (цена по которой необходимо купить/продать бумагу), Покупка (булева переменная в значение true обозначает покупку, false – продажа) и кнопки «Оформить» - для подтверждения оформления заявки.

Характеристики команды, продукта и проекта

Состав компонентов ПО

Разработанное ПО состоит из трех компонентов:

— Первый компонент составляет по объему примерно 15% программного кода и будет написан на **SQL**;

— второй (около 60% кода) — на **C#**;

— третий в объеме 25% кода — на **Java**.

Характеристики продукта

1) Обмен данными — 5
2) Распределенная обработка — 5
3) Производительность — 3
4) Эксплуатационные ограничения по аппаратным ресурсам — 2

5) Транзакционная нагрузка — 3
6) Интенсивность взаимодействия с пользователем (оперативный ввод данных) — 4
7) Эргономические характеристики, влияющие на эффективность работы конечных пользователей — 1
8) Оперативное обновление — 4
9) Сложность обработки — 4
10) Повторное использование — 0
11) Легкость инсталляции — 1
12) Легкость эксплуатации/администрирования — 2
13) Портируемость — 2
14) Гибкость — 2

Организационные и процессные факторы

Для реализации проекта была сформирована новая команда разработчиков, у отдельных членов которой имеется некоторый опыт создания систем подобного типа. В целях сплочения команды были проведены определенные мероприятия, что обеспечило на старте проекта приемлемую коммуникацию внутри коллектива. Заказчик не настаивает на жесткой регламентации процесса, однако график реализации проекта довольно жесткий. Несмотря на то, что предметная область является для разработчиков относительно новой, анализу архитектурных рисков было уделено лишь некоторое внимание. Организация только начинает внедрять методы управления проектами и формальные методы оценки качества процесса разработки.

Оценки по модели COSOMO / факторы проекта

Фактор	**Оценка**
Надежность и уровень сложности (RCPX)	очень высокие
Повторное использование компонентов (RUSE)	не предусматривается
Возможности персонала (PERS)	средние
Опыт работы в разработке систем подобного типа (PREX)	низкий
Сложность платформы (PDIF)	высокая
Интенсивность использования инструментальных средств поддержки (FCIL)	очень интенсивное
Жесткость графика (SCED)	жесткий график

Метод функциональных точек

Функциональная точка - это единица измерения, выражающая объем бизнес-функциональности которую программная система предоставляет пользователю. Функциональность программы связана с обработкой информации по запросу пользователя и независит от применяемых технических решений. Пользователи — это отправители и целевые получатели данных, ими могут быть как реальные люди, так и смежные интегрированные программно-технические системы.

Определение числа функциональных точек является методом количественной оценки размера и сложности программной системы с точки зрения функций, которые система предоставляет пользователю.

Функциональные точки являются мерой оценки функциональности, т.е. полезности программного средства (ПС) с позиции пользователя, то общая функциональность измеряется путем:

— анализа логических групп данных, которые используются и поддерживаются приложением и характеризуют функциональность данных

— анализа вводимой и выводимой пользователем информации, т.е. функциональности совершаемых транзакций

Для этого выделяются 5 типов транзакций и данных:

— Внешний ввод (EI, транзакция, получающая данные от пользователя) — элементарный процесс, перемещающий данные из внешней среды в приложение. Данные могут поступать с экрана ввода или из другого приложения. Данные могут содержать как управляющую, так и деловую информацию. Обрабатываемые данные могут соответствовать одному или нескольким внутренним логическим файлам.

— Внешний вывод (EO, транзакция передающая данные пользователю) — элементарный процесс, перемещающий данные, вычисленные в приложении, во внешнюю среду и связанный с созданием и/или обработкой выходной информации приложения — выходного отчета, документа, экранной формы.

— Внешний запрос (EQ, интерактивный диалог с пользователем, требующий от него каких-либо действий) — элементарный процесс, состоящий из комбинации «запрос/ответ», не связанный с вычислением производных данных или обновлением внутренних логических файлов (базы данных).

— Внутренний логический файл (ILF, информация, которая используется во внутренних взаимодействиях системы) — выделяемые пользователем логически связанные группы данных или блоки управляющей информации, которые поддерживаются внутри продукта и обслуживаются через внешние вводы.

— Внешний интерфейсный файл (EIF, файлы, участвующие во внешних взаимодействиях с другими системами) — выделяемые пользователем логически связанные группы данных или блоки управляющей информации, на которые ссылается продукт, но которые поддерживаются вне продукта.

Для оценки сложности этих элементов используются следующие характеристики:

— RET (тип элемента записи) – это распознаваемая пользователем подгруппа элементов данных во внутреннем или внешнем файле или количество различных используемых форматов записей

— DET (тип элемента данных) – это уникальное распознаваемое пользователем, нерекурсивное (неповторяющееся) поле данных

— FTR (файл типа ссылки) – это тип файла, на который ссылается транзакция. FTR

позволяет подсчитать количество различных файлов (информационных объектов) типа ILF и/или EIF модифицируемых или считываемых в транзакции

Общий алгоритм определения количетсва функциональных точек имеет вид:

— определение количества функциональных типов данных для приложения.

— определение количества связанных с каждым функциональным типом элементарных данных (DET), элементарных записей (RET) и файлов типа ссылок (FTR).

— определение сложности (в зависимости от количества DET, RET и FTR).

— подсчет количества функциональных точек приложения.

— корректировка количества функциональных точек с учетом общих характеристик системы.

При этом корретировка проводится по формуле:

$$CFP = FP \cdot \left(0{,}65 + 0{,}01 \cdot \sum_{i=1}^{14} F_i\right),$$

где:

— FP – изначально подсчитанное количество функциональных точек;

— F_i – i-ый коэффициент характеристик системы, принимающий значения от 0 до 5 (нет влияния, случайное, небольшое, среднее, важное, основное), которое выбирается по таблице

№	Системный параметр	Описание
1	Передача данных	Сколько средств связи требуется для передачи или обмена информацией с приложением или системой?
2	Распределенная обработка данных	Как обрабатываются распределенные данные и функции обработки?
3	Производительность	Нуждается ли пользователь в фиксации времени ответа или производительности?
4	Эксплуатационные ограничения	Насколько сильны эксплуатационные ограничения и каков объем специальных усилий на их преодоление?
5	Частота транзакций	Как часто выполняются транзакции (каждый день, каждую неделю, каждый месяц) ?
6	Оперативный ввод данных	Какой процент информации надо вводить в режиме онлайн?
7	Эффективность работы конечных пользователей	Приложение проектировалось для обеспечения эффективной работы конечного пользователя?
8	Оперативное обновление	Как много внутренних файлов обновляется в онлайновой транзакции?
9	Сложность обработки	Выполняет ли приложение интенсивную логическую или математическую обработку?
10	Повторная используемость	Приложение разрабатывалось для удовлетворения требований одного или многих пользователей?
11	Легкость инсталляции	Насколько трудны преобразование и инсталляция приложения?
12	Легкость эксплуатации	Насколько эффективны и/или автоматизированы процедуры запуска, резервирования и восстановления?
13	Количество возможных установок на различных платформах	Была ли спроектирована, разработана и поддержана возможность инсталляции приложения в разных местах для различных организаций?
14	Простота изменений (гибкость)	Была ли спроектирована, разработана и поддержана в приложении простота изменений?

Расчёты

Авторизация

ILF – Таблица базы данных под пользователя, локальный файл для хранения текущего пользователя:

— Таблица БД: DET = 2, RET = 1

— Локальный файл: DET = 2, RET = 1

EI – Ввод логина и пароля:

— Ввод: DET = 4, FTR = 2

EQ – Уведомление об ошибке:

— Уведомление: FTR = 1, DET = 1

Биржевые сводки

ILF – Таблица биржевых сводок в БД:

— Таблица БД: DET = 3, RET = 2 (два вида столбцов)

EO – Таблица по сводкам:

— Таблица: DET = 3, FTR = 1

EI – Запрос на добавление новой сводки:

— Добавление: DET = 4, FTR = 1

Новая заявка

ILF – Таблица биржевых сводок в БД:

— Таблица БД: DET = 4, RET = 4 (4 вида столбцов)

EIF – Запрос к биржевой системе на покупку:

— Запрос: DET = 3, RET = 3

EI – Создание новой заявки

— Добавление: DET = 5, FTR = 2

Заявки

ILF, EIF – те же, что и при создании заявки.

EO – Таблица по заявкам:

— Таблица: DET = 4, FTR = 1

EI – Запросы на изменение и удаление заявки:

— Изменение: DET = 5, FTR = 2

— Удаление: DET = 2, FTR = 2

Итого

— **ILF**: низкий – 4

— **EIF**: низкий – 1

— **EI**: низкий – 3, средний – 2

— **EO**: низкий – 2

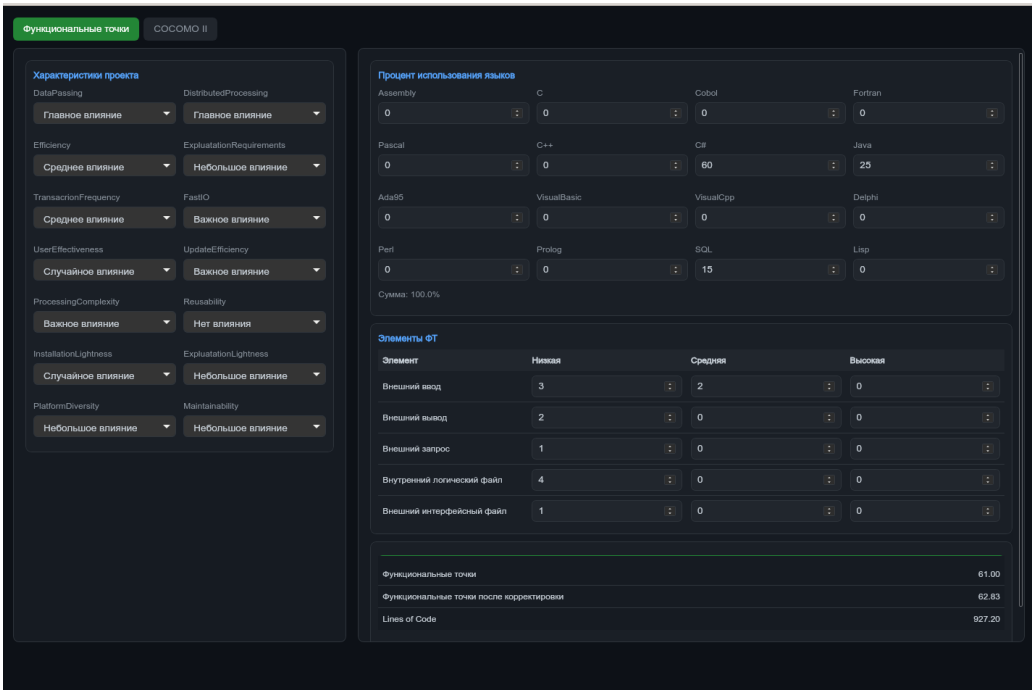
— **EQ**: низкий – 1

С использованием разработанного калькулятора, внеся указанные элементы, характеристики проекта и используемые языки получились следующие значения:

— **Функциональные точки:** 61

— **Функциональные точки после корректировки:** 62.83

— **Строки кода:** 927.20



COCOMO II

Проект COCOMO II стартовал в 1995 году в центре по разработке ПО USC при финансовой и технической поддержке большого количества промышленных предприятий, таких как AT&T, BELL Labs, Hewlett-Packard, Rational, Texas Instruments, Lockheed Martin, Motorola, Xerox и др.

Проект имел триединую задачу:

1) разработать модель для оценки стоимости и сроков создания ПО для того жизненного цикла, который будет применяться в конце 20 – начале 21 века;

2) разработать базу данных стоимости программных проектов и осуществить инструментальную поддержку методов усовершенствования модели стоимости;

3) создать количественную аналитическую схему для оценки технологий создания ПО и их экономического эффекта.

COCOMO II предлагает три различные модели оценки стоимости в COCOMO II

— **Модель композиции приложения** – это модель, которая подходит для проектов, созданных с помощью современных инструментальных средств. Единицей измерения служит объектная точка.

— **Модель ранней разработки архитектуры**. Эта модель применяется для получения приблизительных оценок проектных затрат периода выполнения проекта перед тем как будет определена архитектура в целом. В этом случае используется небольшой набор новых драйверов затрат и новых уравнений оценки. В качестве единиц измерения используются функциональные точки либо KSLOC.

— **Постархитектурная модель** – наиболее детализированная модель COCOMO II, которая используется после разработки архитектуры проекта. В состав этой модели вклю-

чены новые драйверы затрат, новые правила подсчета строк кода, а также новые уравнения.

Модель композиции приложения	Модель ранней разработки архитектуры	Постархитектурная модель
Грубые входные данные	Ясно понимаемые особенности проекта	Детальное описание проекта
Оценки низкой точности	Оценки умеренной точности	Высокоточные оценки
Приблизительные требования	Ясно понимаемые требования	Стабилизировавшиеся основные требования
Концепция архитектуры	Ясно понимаемая архитектура	Стабильная базовая архитектура

Модель композиции приложения

Данная модель используется на ранней стадии конструирования ПО, когда:

— рассматривается макетирование пользовательских интерфейсов;

— оценивается производительность;

— определяется степень зрелости технологии.

Модель ориентирована на применение объектных точек. Объектная точка — средство косвенного измерения ПО. Подсчет количества объектных точек производится с учетом количества экранов (как элементов пользовательского интерфейса), отчетов и компонентов, требуемых для построения приложения.

Правила подсчета объектных точек

— количество изображений на дисплее (экранных форм). Простые изображения принимаются за 1 объектную точку, изображения умеренной сложности принимаются за 2 точки, очень сложные изображения принято считать за 3 точки;

— количество представленных отчетов. Для простых отчетов назначаются 2 объектные точки, умеренно сложным отчетам назначаются 5 точек. Написание сложных отчетов оценивается в 8 точек;

— количество модулей, которые написаны на языках третьего поколения (C, Java, Python и пр.) и разработаны в дополнение к коду, написанному на языке программирования четвертого поколения. Каждый модуль на языке третьего поколения считается за 10 объектных точек.

Сложность экрана зависит от количества:

— представлений на экране;

— таблиц данных, с которыми экран работает;

— серверов, к которым идёт обращение;

— клиентов (пользователей), использующих экран.

Параметр	Простой	Средний	Сложный
Количество представлений (views)	0–1	2–3	≥ 4
Количество таблиц данных	< 3	3–7	> 8
Количество серверов	1	2	≥ 3
Количество клиентов	1–2	3–4	≥ 5

Сложность отчета зависит от количества:

— секций в отчёте;

— таблиц данных, используемых для генерации;

— серверов, откуда берутся данные;

— клиентов, получающих отчёт.

Параметр	Простой	Средний	Сложный
Количество секций	0–1	2–3	≥ 4
Количество таблиц данных	< 3	3–7	> 8
Количество серверов	1	2	≥ 3
Количество клиентов	1–2	3–4	≥ 5

Формула для расчёта трудозатрат и времени разработки имеют вид:

$$\text{NOP} = (\text{Объектные точки}) \times \frac{100 - \%\text{RUSE}}{100},$$

$$\text{Трудозатраты} = \frac{\text{NOP}}{\text{PROD}} \text{ [чел.-мес.]},$$

$$\text{Время} = 3,67 \times (\text{Трудозатраты})^{(0.28+0.2\times(p-1.01))},$$

где:

— RUSE – процент переиспользуемого кода

— p – показатель степени, рассчитывающийся из параметров проекта и команды

— PROD – оценка скорости разработки, берущаяся из таблицы:

Опытность/ возможности разработчика	Зрелость/ возможности среды разработки	PROD
Очень низкая	Очень низкая	4
Низкая	Низкая	7
Номинальная	Номинальная	13
Высокая	Высокая	25
Очень высокая	Очень высокая	50

Значение показателя степени зависит от того, насколько новаторским является данный проект, от гибкости процесса разработки ПО, от применяемых процессов управления рисками, сплоченности команды программистов и уровня управления организацией-разработчиком. Он рассчитывается с учетом пяти показателей по восьми балльной шкале от низшего (7 баллов) до наивысшего (0 баллов) уровня. Значения всех пяти показателей суммируются, сумма делится на 100, результат прибавляется к числу 1.01.

Факторы

— **Новизна проекта (PREC)** – Отражает предыдущий опыт организации в реализации проектов данного типа. Очень низкий уровень этого показателя означает отсутствие опыта, наивысший уровень указывает на компетентность организации-разработчика в данной области ПО.

— **Гибкость процесса разработки (FLEX)** – Отображает возможность изменения процесса разработки ПО. Очень низкий уровень этого показателя означает, что процесс определен заказчиком заранее, наивысший – заказчик определил лишь общие задачи без указания конкретной технологии процесса разработки ПО.

— **Разрешение рисков в архитектуре системы (RESL)** – Отображает степень детализации анализа рисков, основанного на анализе архитектуры системы. Очень низкий уровень данного показателя соответствует поверхностному анализу рисков, наивысший уровень означает, что был проведен тщательный и полный анализ всевозможных рисков.

— **Сплоченность команды (TEAM)** – Отображает степень сплоченности команды и их способность работать совместно. Очень низкий уровень этого показателя означает, что взаимоотношения в команде сложные, а наивысший – что команда сплоченная и эффективная в работе, не имеет проблем во взаимоотношениях.

— **Уровень зрелости процесса разработки (PMAT)** – Отображает уровень развития процесса создания ПО в организации-разработчике.

Расчёты

Объектные точки

Формы:

1) Авторизация – средняя сложность
2) Биржевые сводки – средняя сложность
3) Новая заявка – простая сложность
4) Заявки – простая сложность

Отчёты:

1) Список заявок – средняя сложность
2) Список сводок – простая сложность

Модули:

1) Java – 25%
2) C# – 60%
3) SQL – 15%

Факторы

— **PREC** – Для реализации проекта была сформирована новая команда разработчиков, у отдельных членов которой имеется некоторый опыт создания систем подобного типа – наличие некоторого числа прецедентов

— **FLEX** – Заказчик не настаивает на жесткой регламентации процесса, однако график реализации проекта довольно жесткий – большей частью согласованный процесс

— **RESL** – Несмотря на то, что предметная область является для разработчиков относительно новой, анализу архитектурных рисков было уделено лишь некоторое внимание – некоторое (40%)

— **TEAM** – В целях сплочения команды были проведены определенные мероприятия, что обеспечило на старте проекта приемлемую коммуникацию внутри коллектива – некоторая согласованность

— **PMAT** – рганизация только начинает внедрять методы управления проектами и формальные методы оценки качества процесса разработки – уровень 1+CMM

Среднюю зарплату работника примем как 70000 у.е. в месяц.

Опытность комадны: низкая

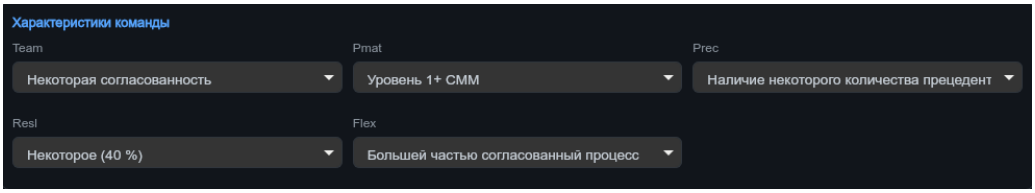
Результаты расчётов:

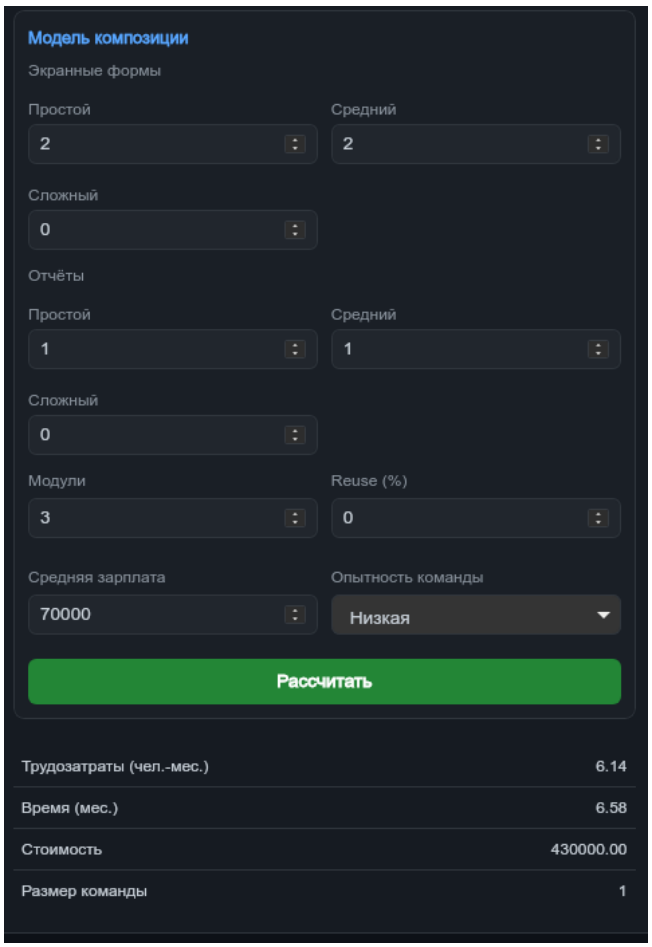
— **Трудозатраты:** 6.14 чел-мес.

— **Сроки:** 6.58 мес.

— **Бюджет:** 430000.00 у.е.

— **Требуемый размер команды:** 1 .





Модель ранней разработки архитектуры

Эта модель применяется для получения приблизительных оценок проектных затрат периода выполнения проекта перед тем как будет определена архитектура в целом.

В данной модели используются следующие формулы:

$$\text{Трудозатраты} = 2,94 \times E_{Arch} \times (\text{Размер})^p$$

где:

— Трудозатраты (работа) — число человеко-месяцев;

— $E_{Arch} = \text{PERS} \times \text{RCPX} \times \text{RUSE} \times \text{PDIF} \times \text{PREX} \times \text{FCIL} \times \text{SCED}$;

— Размер — KSLOC (рассчитываемый, например методом функциональных точек);

— p — показатель степени.

$$\text{Время} = 3,67 \times (\text{Трудозатраты})^{(0.28+0.2\times(p-1.01))}$$

Для расчёта модуля используются следующие драйверы:

— **PERS** — квалификация персонала (Сверхнизкий — аналитики и программисты имеют низшую квалификацию, текучесть больше 45%; Сверхвысокий — аналитики и программисты имеют высшую квалификацию, текучесть меньше 4%);

— **RCPX** — сложность и надежность продукта (Сверхнизкий — продукт простой, специальных требований по надежности нет, БД маленькая, документация не требуется; Сверхвысокий — продукт очень сложный, требования по надежности жесткие, БД сверхбольшая, документация требуется в полном объеме);
— **RUSE** — разработка для повторного использования (Низкий — не требуется; Сверхвысокий — требуется переиспользование в других продуктах);
— **PDIF** — сложность платформы разработки (Сверхнизкий — специальные ограничения по памяти и быстродействию отсутствуют, платформа стабильна; Сверхвысокий — жесткие ограничения по памяти и быстродействию, платформа нестабильна);
— **PREX** — опыт персонала (Сверхнизкий — новое приложение, инструменты и платформа; Сверхвысокий — приложение, инструменты и платформа хорошо известны);
— **FCIL** — оборудование (Сверхнизкий — инструменты простейшие, коммуникации затруднены; Сверхвысокий — интегрированные средства поддержки жизненного цикла, интерактивные мультимедиа коммуникации);
— **SCED** — сжатие расписания (Очень низкий — 75% от номинальной длительности; Очень высокий — 160% от номинальной длительности).

Множитель трудоемкости	Идентификатор	Составные драйверы затрат
Надежность и сложность продукта	RCPX	RELY-DATA-CPLX-DOCU
Повторное использование компонентов	RUSE	RUSE
Сложность платформы	PDIF	TIME-STOR-PVOL
Опытность персонала	PERS	AEXP-PEXP-LTEX
Способности персонала	PREX	ACAP-PCAP-PCON
Возможности среды	FCIL	TOOL-SITE
Сроки	SCED	SCED

	Оценка уровня множителя трудоемкости						
	Сверхнизкий	Очень низкий	Низкий	Номинальный	Высокий	Очень высокий	Сверхвысокий
PERS	2,12	1,62	1,26	1,00	0,83	0,63	0,5
RCPX	0,49	0,60	0,83	1,00	1,33	1,91	2,72
RUSE	n/a	n/a	0,95	1,00	1,07	1,15	1,24
PDIF	n/a	n/a	0,87	1,00	1,29	1,81	2,61
PREX	1,59	1,33	1,22	1,00	0,87	0,73	0,62
FCIL	1,43	1,30	1,10	1,00	0,87	0,73	0,62
SCED	n/a	1,43	1,14	1,00	1,00	1,00	n/a

Расчёты

Драйверы трудоёмкости:
— **PERS** — номинальный
— **RCPX** — очень высокий
— **RUSE** — очень низкий

— **PDIF** — высокий

— **PREX** — низкий

— **FCIL** — очень высокий

— **SCED** — низкий

KLOC – 0.927, посчитанные по методу функциональных точек

Среднюю зарплату работника примем как 70000 у.е. в месяц.

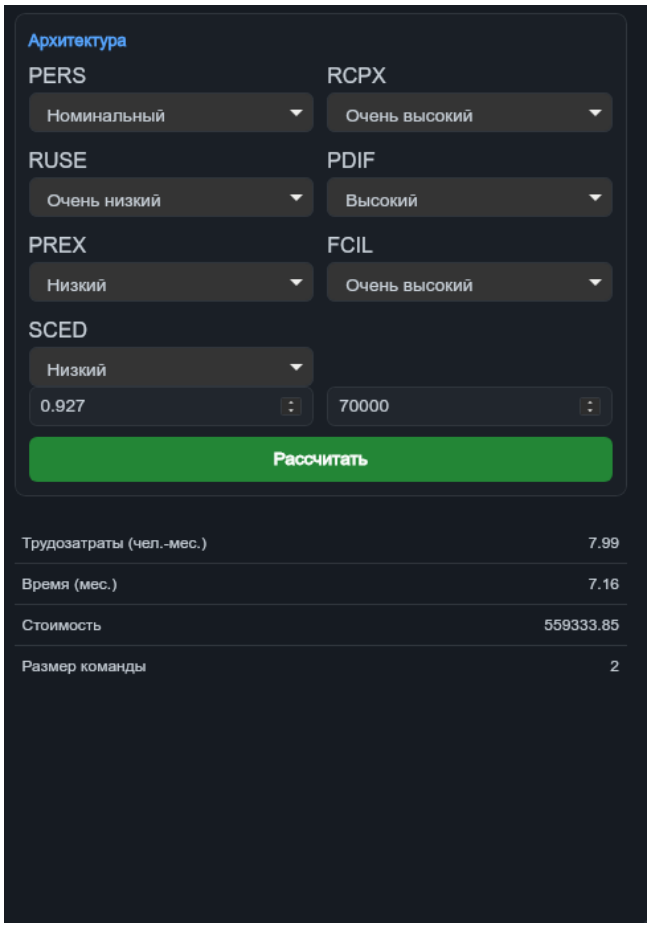
Результаты расчётов:

— **Трудозатраты:** 7.99 чел-мес.

— **Сроки:** 7.16 мес.

— **Бюджет:** 559333.85 у.е.

— **Требуемый размер команды:** 2.



Заключение

В ходе выполнения лабораторной работы были изучены основные подходы к предварительной оценке параметров программных проектов с использованием методики COCOMO II. Практическая работа была выполнена на основе модели композиции приложения и модели ранней разработки архитектуры.

В рамках модели композиции приложения проведён подсчёт объектных точек с учётом сложности экранов и отчётов, а также процентного соотношения языков программирования. С учётом факторов проекта (PREC, FLEX, RESL, TEAM, PMAT) рассчитаны трудозатраты (6,14 чел.-мес.), срок разработки (6,58 мес.) и бюджет (430 000 у.е.). В модели ранней разработки архитектуры на основе KSLOC (0,927) и семи драйверов затрат получены более консервативные оценки: трудозатраты 7,99 чел.-мес., срок 7,16 мес., бюджет 559 333,85 у.е.

Сравнение результатов показало, что модель ранней разработки архитектуры даёт более высокие оценки за счёт учёта большего числа факторов. Применение COCOMO II на ранних этапах позволяет обоснованно планировать стоимость, сроки и ресурсы проекта.